GIT & GITHUB

Session 1 — Technical Report | Group B

Student: Yasmine Boutchiche

GitHub username: yasmine2609

Repository: https://github.com/yasmine2609/git-session1-yasmine2609

Date: 13 October 2026

1. Introduction

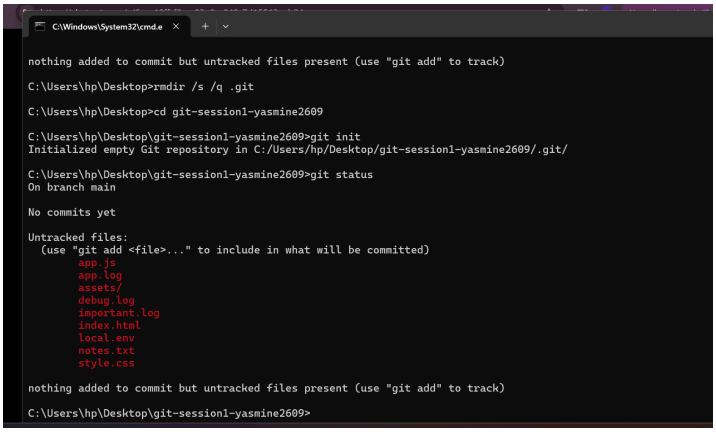
The purpose of this practical work was to learn the basic Git and GitHub workflow: creating a local repository, deciding which files Git should track, recording changes with commits, working on a separate branch, merging changes, publishing the project to GitHub, and synchronizing local and remote versions. Git manages the project history locally, while GitHub hosts the remote repository.

2. Git installation and repository setup

I verified that Git was installed on Windows using **git --version**. I configured my Git name, university email, and the default branch name **main**. I renamed the starter folder to **git-session1-yasmine2609**, entered that folder, and initialized it with **git init**. A check with **git status** showed the project files as untracked.

```
git --version | git config --global --list
```

```
git init | git status
```



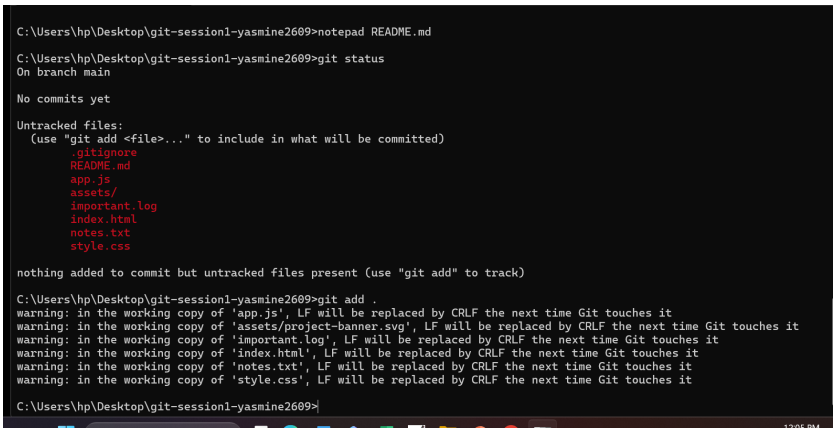
Evidence 1 — Git repository initialized inside the correctly named project folder; initial files appear as untracked.

3. README and .gitignore

I created **README.md** to document the project. I also created **.gitignore** so that the local environment file and ordinary log files would not be tracked, while **important.log** remained included. I verified the ignore rules with **git check-ignore -v**. The command showed that **local.env**, **app.log**, and **debug.log** matched ignore rules. The separate check for **important.log** produced no ignore match.

```
git check-ignore -v local.env app.log debug.log
```

```
git check-ignore important.log
```



Evidence 2 — The ignore rules were verified in Command Prompt.

4. Tracking files and recording commits

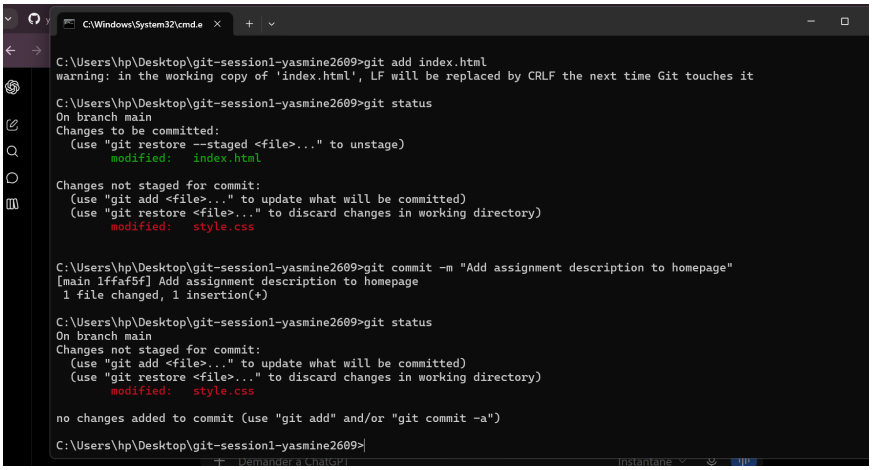
I staged the allowed project files with **git add .**, inspected them with **git status**, and made the initial commit. I then updated **notes.txt** with Git learning goals and committed that change separately. After modifying **index.html** and **style.css**, I staged and committed the changes with descriptive messages. The command **git log --oneline** displayed the recorded checkpoints.

```
git add .
```

```
git commit -m "Initial project setup with starter files"
```

```
git commit -m "Add Git learning goals to notes"
```

```
git log --oneline
```



Evidence 3 — Commit history with four recorded checkpoints and a clean working tree.

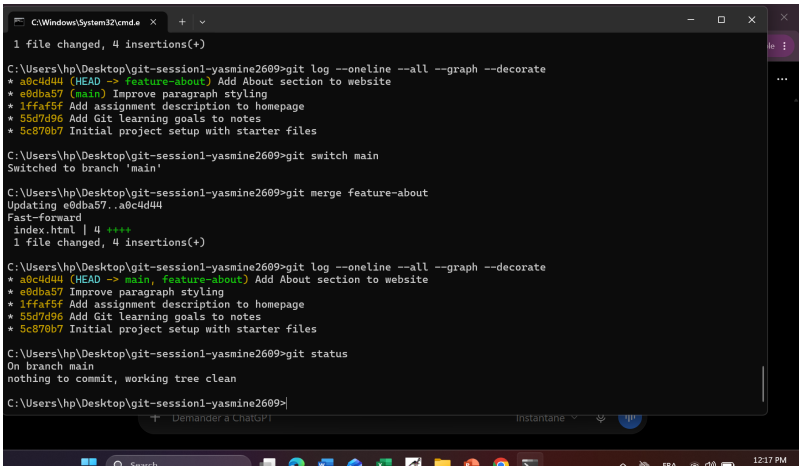
5. Creating and merging a feature branch

I created a branch named **feature-about** and switched to it. I edited **index.html** to add an About section, then committed that change on the feature branch. I switched back to **main** and merged **feature-about**. Git reported a **fast-forward** merge, meaning main advanced to the feature branch commit without a conflict. The history confirmed both branch names at the same commit.

```
git branch feature-about | git switch feature-about
```

```
git commit -m "Add About section to website"
```

```
git switch main | git merge feature-about
```



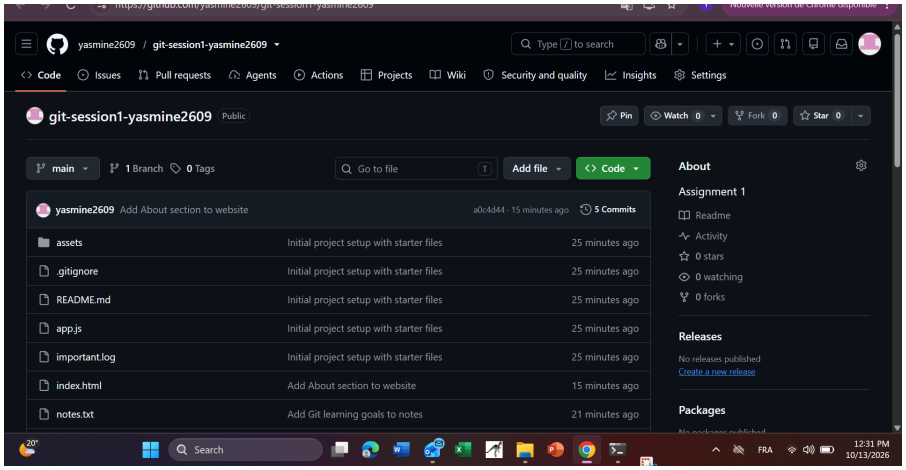
Evidence 4 — Successful fast-forward merge into main and clean status.

6. Connecting the repository to GitHub

I connected the local project to a GitHub repository using the remote name **origin**. I checked the remote address with **git remote -v** and uploaded the main branch with **git push -u origin main**. GitHub authentication was completed through the browser. The terminal confirmed that the new remote branch was created and that local main tracked origin/main. The GitHub website then displayed the project files and five commits.

```
git remote -v
```

```
git push -u origin main
```



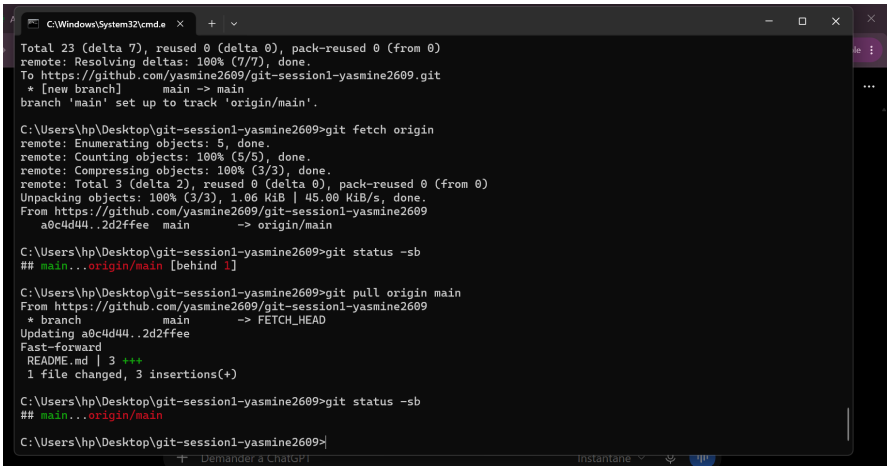
Evidence 5 — GitHub repository page with project files and commit history.

7. Fetching and pulling remote changes

I edited **README.md** directly on GitHub and committed that update online. Back in Command Prompt, I ran **git fetch origin** to download remote information without changing my local files. The command **git status -sb** then showed **[behind 1]**, confirming that GitHub had one commit not yet present locally. I ran **git pull origin main**; Git performed a fast-forward update of README.md. The next status check showed **main...origin/main** without a behind indicator.

```
git fetch origin | git status -sb
```

```
git pull origin main
```



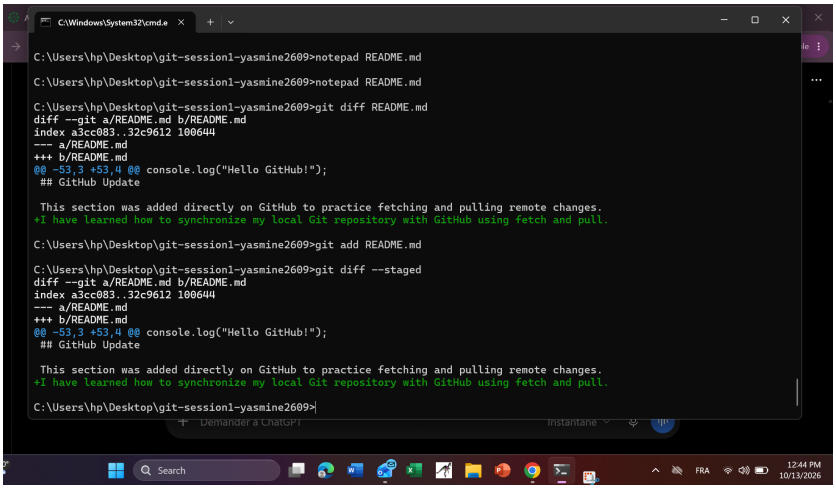
Evidence 6 — The remote README update was pulled successfully and the branches were synchronized.

8. Inspecting and staging a final local modification

After verifying that the GitHub sentence appeared in the local README, I added one more sentence explaining what I learned about fetch and pull. I ran **git diff README.md** to inspect the change before staging. The added line appeared with a plus sign. Next, I ran **git add README.md** and **git diff --staged** to confirm exactly what was prepared for the next commit. The screenshot shows the change staged; it does not show a new commit or push after this point.

```
git diff README.md
```

```
git add README.md | git diff --staged
```



Evidence 7 — Final README change inspected before and after staging.

9. Conclusion

This session helped me understand how Git tracks file changes and why commits are useful as project checkpoints. I practiced using a feature branch without immediately changing main, merging the branch, connecting a local repository to GitHub, and using fetch and pull to synchronize changes. The final demonstrated state has the latest README modification staged for a possible next commit.

Submission repository: https://github.com/yasmine2609/git-session1-yasmine2609